Building Resilient Distributed Systems

Muhammad Usman Gillani | Student ID: bsai23062

Part 1: Distributed Systems Bug Analysis (StudySync Scenario)

1.1 The Synchronization Bug (Lost Update Anomaly)

In the initial StudySync collaborative document editor, concurrent updates are prone to a severe data loss pattern known as the **Lost Update Anomaly**. The root cause is a complete lack of concurrency control or isolation boundaries at the persistence and application layer. When User A and User B concurrently decide to edit the same lecture notes, they both perform an independent and overlapping **Read-Modify-Write cycle** simultaneously. Under naive operations, User A reads document state S at timestamp T_1. Fractions of a second later, User B reads the identical state S at timestamp T_2. User A completes their edits locally and submits their payload, which the backend blindly writes, updating the database state to S'. Immediately following, User B completes their independent edit (having no awareness of User A's intermediate submission) and submits their payload. The server receives User B's write and blindly overwrites the database with User B's state $S''. Consequently, User A's entire set of changes is permanently obliterated without warning. This is the classic lost update anomaly, arising because neither the application code nor the database enforces atomic version checks or transactional optimistic/pessimistic locking mechanisms.

1.2 The Webhook Coordination Bug (Unreliable Message Delivery)

In the subscription cancellation flow, Clerk is configured to dispatch a single HTTP POST event notifying the StudySync backend of a cancellation. The fundamental defect is a coordination failure over an unreliable network. Clerk uses an *at-most-once* delivery paradigm in its native, non-retry configurations. Because HTTP is an unreliable, stateless protocol, packet drops, routing loops, or transient server-side crashes will prevent the webhook from being processed. Since the initial implementation contains no transaction deduplication, retry queues, or acknowledgment handshakes, a dropped HTTP request is permanently lost. The Clerk billing provider registers the user as 'Cancelled' (state X), while the local database continues to mark the user as 'Premium' (state Y), creating a permanent state inconsistency. This coordination mismatch permits cancelled users to bypass paywalls indefinitely, representing a significant loss of business integrity.

1.3 The Fault Tolerance Bug (LLM API Timeout Cascading Failure)

The third defect is an architectural fault tolerance failure caused by wrapping slow, external, synchronous LLM calls directly inside a web service endpoint without isolating execution pools or establishing fail-fast thresholds. When an external LLM provider hangs, a single synchronous call ties up a FastAPI async worker thread for the entire 60-second timeout period. FastAPI handles incoming concurrent requests using a finite worker pool (or a thread pool for synchronous database/IO tasks). Under moderate traffic, if enough users concurrently hit the AI feature while the LLM is down, all available async workers and thread pools are quickly exhausted, blocking on the hanging downstream network sockets. As a result, the entire web server is rendered completely unresponsive, and even unrelated, instantaneous operations (like retrieving static notes or registering users) fail with Gateway Timeouts. This allows a slow, optional third-party service to act as a **Single Point of Failure (SPOF)**, cascading a localized dependency delay into a total system-wide blackout.

Part 2: Resilient Distributed System Architecture Designs

2.1 Synchronization Fix — Optimistic Locking

To resolve the Lost Update Anomaly, we implement **Optimistic Locking** using a dedicated version column in the database. Optimistic locking is an ideal concurrency control mechanism for read-heavy, low-contention environments like document editing. It avoids the heavy performance penalties and potential deadlock scenarios associated with pessimistic database row locks (e.g., SELECT ... FOR UPDATE).

Database Schema and Atomic Query:

```
ALTER TABLE documents ADD COLUMN version INTEGER DEFAULT 1;

/* The atomic write query executed on save */
UPDATE documents
SET content = :new_content, version = version + 1
WHERE id = :doc_id AND version = :client_read_version;
```

When a client requests a document (GET), the server returns the content alongside its current version integer. When submitting modifications, the client must send back that version number inside the payload. The database then executes the update query above. If another concurrent session (User A) has successfully saved its changes first, the database version will have already incremented, making User B's query criteria WHERE version = 5 fail. No rows are modified. The backend detects that zero rows were updated, rolls back any associated changes, and returns an HTTP **409 Conflict** response. This alerts the client that a conflict occurred, prompting them to fetch the latest changes, resolve conflicts, and retry.

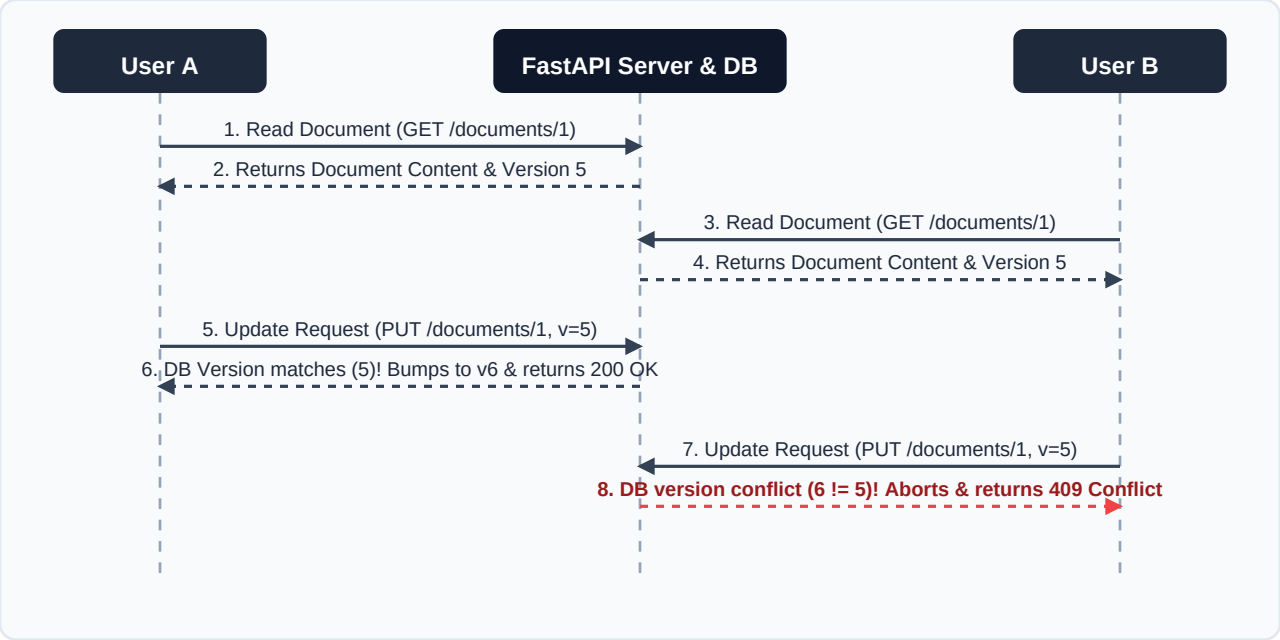

Figure 1: Sequence diagram of concurrent edit resolution using version-based optimistic locking

2.2 Webhook Coordination Fix — Idempotent Webhook Handler & Retries

To ensure reliable and consistent webhook state coordination between Clerk and StudySync, we establish an **Idempotent Webhook Handler**. We create a processed_webhooks table with a unique constraint on Clerk's event_id (provided in the webhook metadata). When an event is received, the backend executes the following atomic flow inside a single database transaction:

1. Check if event_id already exists in the processed_webhooks database table. If yes, immediately abort and return a friendly **HTTP 200 OK** response (duplicate event is discarded safely without modifying state).
2. If the event is new, execute the business logic (e.g., set is_premium = False) and insert the event_id into the processed_webhooks table.
3. Commit the transaction. If a race condition occurs, the unique database constraint will abort the duplicate execution safely.

In addition, we configure Clerk's webhook dashboard to implement **exponential backoff retries**. If our server is down, Clerk will continuously retry the cancellation. Once our server recovers, the first retry processes the event, and any redundant retries are safely ignored by our idempotent handler. For maximum durability, failed events after retries can be pushed to a **Dead-Letter Queue (DLQ)** in Redis, allowing automatic alerts and manual administrative replays.

2.3 Fault Tolerance Fix — Three-State Circuit Breaker Pattern

To isolate slow AI processes and prevent application worker pool starvation, we wrap the downstream LLM service inside a **Circuit Breaker**. The breaker tracks the state of the LLM using a state machine consisting of three operational states:

• **CLOSED:** All requests flow directly to the LLM. Every successful call clears the failure count. Every exception or timeout increments the failure counter. If failures exceed a defined threshold (e.g., 5 consecutive failures), the breaker trips to the OPEN state.

• **OPEN:** Incoming requests are immediately short-circuited. No socket connections are made to the LLM, protecting server-side async resources and worker threads from hanging. The server instantly returns a predefined **degraded fallback response** (e.g., an HTTP 503 status code with a payload stating 'AI suggestions are temporarily unavailable') in under 10 milliseconds. A cooldown timer (e.g., 30 seconds) is initiated upon entering the OPEN state.

• **HALF-OPEN:** Once the cooldown timer expires, the breaker transitions to the HALF-OPEN state. It permits a single 'trial' request to pass through. If this trial call succeeds, the breaker assumes the downstream service has recovered, resets the failure counter, and returns to the CLOSED state. If the trial call fails, the breaker assumes the service is still unhealthy, immediately returns to the OPEN state, and restarts the cooldown timer.

2.4 Distributed Systems CAP Theorem Trade-offs

The architectural mitigations represent a classic study of trade-offs defined by the **CAP Theorem**. Our version-based **Optimistic Locking** design explicitly prioritizes **Consistency (C) over Availability (A)**. When concurrent write conflicts are detected, the system deliberately rejects edits (degrading write availability for the conflicting client) to guarantee that the database remains in a perfectly accurate, uncorrupted, and consistent state. Conversely, the **Circuit Breaker** pattern prioritizes **Availability (A) over Consistency (C)** (specifically in terms of query up-to-dateness or suggestion completeness). When the downstream AI service fails, the circuit breaker bypasses the LLM and immediately serves a stale, cached, or degraded placeholder fallback response. This ensures the application remains fully available and highly responsive for the end user, prioritizing usability and operational uptime over real-time computational accuracy.